WEEK 3 — AGENTS

Build an AI agent that does real work for you

ابنِ وكيل ذكاء اصطناعي يسوّي شغلاً حقيقياً عنك

YOUR AI TOOL

Codex / Cursor / Copilot

Tuned for coding agents — keep each step as a task comment and let the agent execute file-by-file, checking the box before moving on.

مهيّأة لوكلاء البرمجة — خلّ كل خطوة تعليق مهمة ودع الوكيل ينفّذ ملف بملف ويعلّم كل خطوة قبل التالية.

YOUR SYSTEM · POWERSHELL

Windows

Commands use PowerShell. Prefer WSL2 (Ubuntu) when a step calls for native Linux tooling.

الأوامر بـ PowerShell. استخدم WSL2 (أوبونتو) عند الحاجة لأدوات لينكس الأصلية.

The Agent Builder Blueprint: From Idea to a Worker That Actually Ships

ابدأ صغيراً: الوكيل الجيد ليس عقلاً عبقرياً، بل حلقة بسيطة تخطط ثم تنفذ ثم تلاحظ، محاطة بأدوات محدودة وحواجز أمان واضحة. هذا المخطط يأخذك من اختيار مهمة تستحق الأتمتة إلى وكيل تشغّله وتراقبه بثقة. القاعدة الذهبية: ابنِ الأصغر شيء ممكن، اختبره بقسوة، ثم وسّعه.

A useful agent is not a magic brain. It is a tight loop wrapped around a few well-chosen tools, fenced in by guardrails, and judged by a scorecard you trust. Most agent projects fail not because the model is weak but because the *job* was vague, the tools were unsafe, or nobody checked whether the thing actually worked. This blueprint fixes that order: pick the right job, design the loop, wire tools safely, fence it, evaluate it, then ship and watch it.

Pick a Job Worth Automating

Do not start with "let's build an agent." Start with a recurring, painful, well-bounded task. The best first candidates share four traits: they happen often, they follow a checkable procedure, a wrong answer is cheap to catch, and a human currently does them on autopilot.

Score each candidate before committing:

1. **Frequency** — does it happen daily or weekly? Rare tasks rarely repay the build cost.
2. **Verifiability** — can you tell within seconds whether the output is right? If not, you cannot evaluate the agent.
3. **Blast radius** — what breaks if it does the task wrong? Prefer reversible, low-stakes work for v1.
4. **Boundedness** — can you describe "done" in one sentence? Open-ended goals produce wandering agents.

The *why*: an agent compounds whatever process you give it. Automating a fuzzy, high-stakes, one-off task multiplies risk without multiplying value. Triaging inbound support tickets, summarizing a folder of documents, or reconciling two data exports are ideal starters — frequent, checkable, and reversible.

Design the Agent Loop: Plan, Act, Observe

Every capable agent runs the same heartbeat: it forms a plan, takes one action through a tool, observes the result, and decides whether it is done. Keep this loop explicit rather than hoping the model improvises it.

A reliable loop has these stages:

- **Plan** — restate the goal, list the next concrete step, and name the tool it will use.
- **Act** — call exactly one tool with arguments it can justify.
- **Observe** — read the tool's output, including errors, and update its understanding.
- **Decide** — continue, change approach, ask a human, or finish with a final answer.

Bound the loop with a hard step limit and a stop condition. An agent without a ceiling will spin forever on an impossible task and burn money. The *why*: separating planning from acting makes failures legible — you can see whether the agent chose the wrong step or the right step failed, which is the difference between a prompt bug and a tool bug.

```
SYSTEM: You operate in a strict loop. Each turn you output exactly one block:
THOUGHT: what you know and what is still missing
PLAN: the single next step and why it moves toward the goal
```

```
ACTION: one tool call with arguments, OR "FINISH" with the final result
After each ACTION you will receive an OBSERVATION. Never assume a tool's
result — wait for the OBSERVATION. Stop after 12 steps or when the goal's
success condition is met. If blocked twice on the same step, escalate to a human.
```

Give It Tools and Capabilities Safely

An agent is only as capable as its tools, and only as safe as their narrowest version. Design each tool as a small, single-purpose function with a strict input schema and a predictable output. Avoid one god-tool that "does anything"; it removes your ability to reason about what the agent can do.

For each tool, define:

- **A precise name and one-line purpose** the model can match to intent.
- **A typed input schema** with validation, so malformed calls fail loudly instead of doing something surprising.
- **A scoped capability** — read-only where possible; write access only where truly needed.
- **A structured result**, including a clear error shape the agent can read and react to.

The *why*: agents pick tools by their descriptions, so vague descriptions cause wrong calls. Narrow, validated tools also shrink the attack surface — the agent simply cannot perform actions you never exposed. Treat every tool description as part of the prompt, because it is.

```
TOOL: search_records
PURPOSE: Find records matching a query. READ-ONLY. Never modifies data.
INPUT: { "query": string (required), "limit": integer 1-50 (default 10) }
OUTPUT: { "results": [ {id, title, summary} ], "count": integer }
ON_ERROR: { "error": string, "retryable": boolean }
RULES: Use this before any update tool. Do not invent record IDs;
only use IDs returned by this tool.
```

Guardrails and Permissions

Guardrails are what let you trust an agent in production. Build them in three layers: what it *can* do, what it *must confirm*, and what it *can never* do. Permissions belong in code, not in the prompt — a model can be talked out of a prompt rule, but it cannot bypass a function that simply refuses.

Practical controls worth adding from day one:

- **Allow-lists** for tools and destinations; deny everything not explicitly permitted.
- **Human-in-the-loop gates** for any irreversible or high-value action (deleting, paying, emailing customers).
- **Rate and budget caps** on tool calls, tokens, and total run cost, enforced outside the model.
- **Input and output filtering** so untrusted content cannot smuggle in new instructions, and secrets never leave in responses.

The *why*: the model is a suggestion engine, not a security boundary. Assume any input it reads might try to redirect it, and assume it will occasionally choose wrongly. Hard limits in your own code turn "it usually behaves" into "it cannot misbehave beyond this line."

Test and Evaluate It

You cannot improve what you cannot measure, and "it looked fine" is not measurement. Build an evaluation set before you tune anything: a collection of real inputs paired with known-good outcomes or at least a checkable success condition.

A workable evaluation routine:

1. **Collect 20–50 representative cases**, including the messy and adversarial ones, not just the easy path.
2. **Define a pass condition per case** — exact match, a checker function, or a rubric a reviewer applies consistently.
3. **Run the whole set on every change** and track pass rate, average steps, and cost per task.
4. **Save every failure** as a permanent regression case so a fixed bug never returns silently.

The *why*: agents are non-deterministic, so a single good demo proves nothing. A scored suite turns vibes into a number you can watch move. When you tweak a prompt or swap a tool, the suite tells you whether you helped or quietly broke three other cases.

Ship and Monitor

Shipping is the start of the work, not the end. Roll out behind a flag to a small slice of traffic, keep a human reviewing outputs at first, and widen the gate only as the numbers hold. Log everything: every plan, tool call, observation, and final result, because when something goes wrong you will need the full trace to understand why.

Operational essentials:

- **Full run traces** stored and searchable, so any incident is reconstructable.

- **Live metrics** for success rate, escalation rate, latency, and cost — with alerts on regressions.
- **A kill switch** that disables the agent or a specific tool instantly, without a deploy.
- **A feedback path** where flagged outputs flow straight back into the evaluation set.

The *why*: real traffic always finds inputs your test set missed. Monitoring closes the loop — production failures become tomorrow's regression cases, and the agent gets measurably safer over time instead of drifting unnoticed.

Checklist

- [ ] The job is frequent, verifiable, reversible, and describable in one sentence.
- [ ] The loop is explicit (plan, act, observe, decide) with a hard step ceiling.
- [ ] Every tool is single-purpose, schema-validated, and read-only unless write is essential.
- [ ] Permissions are enforced in code via allow-lists, not just prompt instructions.
- [ ] Irreversible actions require human confirmation; budget and rate caps are in place.
- [ ] An evaluation set of 20+ real cases runs on every change with a tracked pass rate.
- [ ] Every fixed failure is saved as a permanent regression case.
- [ ] Full run traces, live metrics, alerts, and a one-click kill switch exist before wide rollout.
- [ ] Rollout is gradual, behind a flag, with human review early on.